StormCare RAG: A Reproducible Retrieval-Augmented and Parameter-Efficient NLP Pipeline for Storm-Response Documents

Adrija Ghosh[1] (agd9c@umsystem.edu)

***Abstract*— Natural disasters generate large volumes of technical reports, emergency guidance, forecasting documents, and public-safety materials that are often difficult to search and analyze efficiently. This paper presents StormCare RAG, a compact storm-domain NLP pipeline for PDF-based hurricane and emergency-response corpora. The system performs PDF ingestion, paragraph-level chunking, custom Byte Pair Encoding tokenizer training, tokenizer diagnostics, BM25 retrieval, next-token language-model smoke training, and a controlled comparison between a fully trainable base model and a parameter-efficient LoRA-adapted model. Evaluation included tokenizer coverage, retrieval quality, training loss, held-out test loss, perplexity, and trainable-parameter efficiency. Results showed strong tokenizer coverage with an unknown-token rate of 0.0, effective BM25 retrieval, and stable next-token training. While the base model achieved lower test loss and perplexity, LoRA reduced trainable parameters by approximately 99.35 percent. These findings suggest that parameter-efficient adaptation can provide major computational savings in compact-model settings, though not always improved held-out performance. The primary contribution of this work is a reproducible end-to-end workflow for storm-domain document processing, retrieval, adaptation, and evaluation.**

I. INTRODUCTION

A. Problem Motivation

Hurricanes and major storms create urgent information needs. Emergency managers, researchers, students, and community organizations often need to understand flooding risks, evacuation guidance, emergency supplies, infrastructure damage, storm-surge behavior, and forecasting uncertainty. Much of this information exists in PDF reports, government documents, technical papers, and response guides. Although these documents may contain valuable knowledge, they are not always easy to use directly because they are lengthy, heterogeneous, and written in formal or technical language.

A practical storm-response NLP system should be able to ingest raw documents, divide them into meaningful text chunks, retrieve relevant evidence, and support downstream language-modeling or adaptation experiments. This project addresses that need through a compact but complete pipeline called StormCare RAG.

B. Task Definition

The task is to build and evaluate an end-to-end storm-domain NLP pipeline that can:

Convert raw PDF documents into structured text chunks. Train and evaluate a custom tokenizer on the storm corpus. Retrieve relevant chunks using keyword-based retrieval. Train a small next-token language model as a smoke-test language-modeling task. Compare a fully trainable base model with a LoRA-adapted parameter-efficient model. Package the workflow into a reproducible project structure with scripts, result files, plots, and dashboard orchestration.

C. Why the Problem Matters

Disaster-related documents often contain life-critical information. If users cannot quickly find relevant guidance, they may miss important details about evacuation, flooding, emergency supplies, or infrastructure risk. Retrieval-based systems can help make long documents more usable by returning focused evidence passages. Similarly, domain adaptation can help language models better reflect specialized terminology, such as storm surge, evacuation orders, emergency kits, risk communication, and forecasting uncertainty.

This problem also matters educationally. A compact, reproducible system helps demonstrate how modern NLP systems are built from multiple components: data ingestion, tokenization, retrieval, model training, adaptation, evaluation, and reporting. StormCare RAG therefore serves both as a disaster-domain prototype and as a structured learning project.

D. Summary of Approach and Findings

The system integrates PDF ingestion, JSONL chunk construction, custom BPE tokenizer training, tokenizer diagnostics, BM25 retrieval, next-token LM smoke training, and a controlled base-versus-LoRA experiment. The tokenizer achieved an unknown-token rate of 0.0 across the diagnostic sample, suggesting strong corpus coverage. BM25 retrieval returned relevant storm-response chunks for queries such as "flooding storm surge," "evacuation orders," and "emergency kits water batteries." The LM smoke test showed stable training, with train loss decreasing from 5.7216 at step 0 to 4.9590 at step 79. In the final comparison, the base model achieved a test loss of 4.8625 and test perplexity of 129.3472, while the LoRA-adapted model achieved a test loss of 5.3904 and perplexity of 219.2973. However, LoRA used only 3,072 trainable parameters compared with 470,528 for the base model, reducing trainable parameters by approximately 99.35

II. RELATED WORK / BACKGROUND

Retrieval-Augmented Generation, commonly known as RAG, combines information retrieval with language generation [3]. Instead of relying only on the model's internal parameters, a RAG-style system first retrieves relevant external documents or chunks and then uses those chunks to support downstream answering, summarization, or reasoning. This

design is useful when the corpus contains domain-specific or frequently updated information.

Traditional keyword retrieval methods such as BM25 remain strong baselines for document search [1]. BM25 scores documents based on term frequency, inverse document frequency, and document-length normalization. Although neural dense retrieval methods can capture semantic similarity, BM25 is simple, interpretable, and effective for small or medium-sized technical corpora.

Tokenizer design is another important component of NLP systems. Byte Pair Encoding and related subword methods help represent words as smaller units [2], allowing a model to handle rare or domain-specific terms. In this project, a custom BPE tokenizer was trained directly on the storm-response corpus. This decision allowed the tokenizer to reflect the project's vocabulary rather than relying entirely on a general-purpose tokenizer.

Parameter-Efficient Fine-Tuning methods such as LoRA are commonly used [4] to adapt models while training only a small number of additional parameters. LoRA introduces low-rank trainable matrices into selected model components. This can greatly reduce memory and compute requirements. However, LoRA performance depends on the target modules, model size, dataset size, training duration, and adaptation objective. In this project, LoRA was applied as a controlled comparison rather than as a guaranteed improvement.

StormCare RAG is positioned as an engineering-integration project[6]. It does not claim to introduce a new retrieval algorithm or a new language-model architecture. Instead, its contribution is the construction of a complete, reproducible workflow that connects document ingestion, tokenizer training, retrieval, language modeling, adaptation, and evaluation in one compact system.

III. Data / Dataset / Corpus

A. Data Source and Construction

The corpus was constructed from storm-domain PDF documents related to hurricanes, forecasting, emergency response, and disaster preparedness. The pipeline begins by ingesting PDF files and converting their extracted text into structured JSONL chunks. Each chunk preserves useful metadata such as source document, page number, and chunk identifier where available. This design supports both retrieval and later evidence inspection.

The implemented system includes a PDF-to-JSONL conversion script.

The broader project also includes tokenizer training, tokenizer diagnostics, retrieval, smoke testing, base-versus-PEFT comparison, dashboard orchestration, and final packaging scripts.

B. Preprocessing Decisions

The preprocessing pipeline used paragraph-oriented chunking with a minimum character threshold. This helped avoid extremely small chunks that may not contain enough semantic context. The purpose of chunking was to create retrieval-ready units that were short enough to search effectively but long enough to preserve meaningful disaster-response information.

TABLE I
Major Preprocessing Steps

Step	Description	Purpose
PDF ingestion	Extract text from storm-domain PDF files	Convert raw documents into machine-readable text
Paragraph chunking	Split documents into manageable text units	Support retrieval and training
JSONL formatting	Store chunks with metadata	Improve reproducibility and traceability
Tokenizer training	Train a custom BPE tokenizer	Adapt tokenization to domain vocabulary
Diagnostics	Measure token coverage and token density	Validate tokenizer behavior

C. Train / Validation / Test Strategy

For the language-modeling experiment, the corpus was divided into train, validation, and test or held-out splits. The same held-out evaluation strategy was used for both the base model and the LoRA-adapted model. This was important because the project aimed to make a fair comparison between full training and parameter-efficient adaptation.

TABLE II
Controlled Base vs LoRA Comparison Conditions

Fairness Control	Description
Same task	Both systems used next-token language modeling
Same split	Both were evaluated on the same held-out data
Same metrics	Test loss and test perplexity were used
Same evaluation procedure	Both systems were evaluated consistently
Same seed/shared initialization	The comparison used controlled initialization conditions

These controls made the base-versus-adapted comparison more reliable.

D. Licensing, Privacy, Bias, and Ethics

Because the corpus is based on storm-response documents, responsible data use is important. Public disaster documents may still include sensitive community information, descriptions of vulnerable populations, or disaster impacts. The system should therefore be used carefully and should not be treated as a replacement for official emergency guidance.

IV. Method

A. System Overview

StormCare RAG is organized as a modular NLP pipeline. Each stage produces artifacts that can be inspected independently. This architecture emphasizes reproducibility. Each component can be run using a separate script, and the final dashboard helps organize outputs and submission artifacts.

TABLE III

ETHICAL CONSIDERATIONS AND MITIGATION STRATEGIES

Concern	Relevance to This Project	Mitigation
Privacy	Disaster reports may mention affected communities	Avoid exposing personal or sensitive details
Bias	Documents may over-represent some regions or storm types	Clearly describe corpus scope
Reliability	Retrieved text may be incomplete or outdated	Cite source documents and use official guidance when needed
Misuse	System outputs could be mistaken for emergency instructions	Include deployment caveats and human review
Accessibility	Emergency information must be understandable	Encourage clear summaries and source-backed outputs

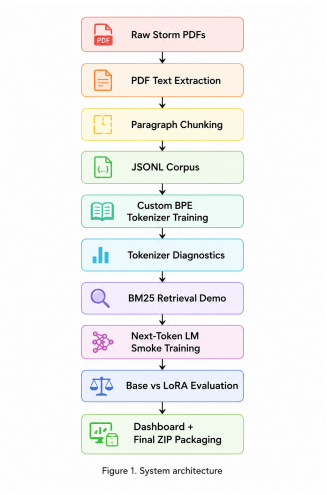

Fig. 1. System Architecture

B. Model Choice

The modeling task was framed as next-token language modeling. A compact causal language model was used for smoke training and base-versus-adapted comparison. The goal was not to train a production-scale model, but to verify that the project pipeline could support end-to-end training, evaluation, and adaptation.

The base model was fully trainable, while the adapted model used LoRA to reduce the number of trainable parameters. This comparison allowed the project to study the tradeoff between model quality and parameter efficiency.

C. Tokenizer Choice

TABLE IV

TOKENIZER DIAGNOSTIC RESULTS

Metric	Value
Number of documents	91
Average tokens per document	605.4835
Tokens per 1,000 characters	445.1941
Unknown-token rate	0.0

A custom BPE tokenizer was trained on the storm corpus. This was appropriate because storm-response documents include domain-specific terms such as evacuation, storm surge, flooding, emergency kits, infrastructure, forecasting, and disaster response. A custom tokenizer can better represent frequent domain patterns than a generic tokenizer in a compact project setting.

The unknown-token rate of 0.0 suggests that the tokenizer had strong coverage on the diagnostic sample.

D. Training and Adaptation Setup

The base model and LoRA-adapted model were evaluated using the same next-token prediction task. The LoRA configuration used:

TABLE V

LORA CONFIGURATION SETTINGS

LoRA Setting	Value
Rank	8
Alpha	16
Dropout	0.05
Target module	`lm_head`

The LoRA setup was intentionally lightweight. It trained far fewer parameters than the base model, making it useful for studying parameter efficiency under constrained conditions.

E. Retrieval and Orchestration Components

BM25 retrieval was used as the retrieval baseline. The retrieval system searched over the chunked storm corpus and returned ranked chunks with metadata. This made retrieval outputs interpretable and inspectable.

The orchestration layer included a dashboard script:

`training_code/m3_dashboard.py`

The dashboard supported artifact checks and final ZIP generation, making the project easier to reproduce and submit.

V. EXPERIMENTAL SETUP

A. Baselines

The project used two major baselines, shown in Table VI.

TABLE VI

EXPERIMENTAL BASELINES

Baseline	Purpose
BM25 retrieval	Simple, interpretable retrieval baseline
Fully trainable base LM	Comparison point for LoRA adaptation

The adapted system was the LoRA-based model. It was compared against the base model using the same held-out evaluation conditions.

B. Metrics

The evaluation used both retrieval and language-modeling metrics, as summarized in Table VII.

TABLE VII
EVALUATION METRICS

Component	Metric	Purpose
Tokenizer	Unknown-token rate	Measures tokenizer coverage
Tokenizer	Tokens per 1,000 characters	Measures tokenization density
Retrieval	Top-1 BM25 score	Measures query-match strength
LM training	Train/validation/test loss	Measures learning stability
Final evaluation	Test loss	Measures held-out next-token performance
Final evaluation	Test perplexity	Interpretable LM quality metric
Efficiency	Trainable parameters	Measures adaptation cost

C. Implementation Details

The project was implemented as a reproducible Python workflow. The primary scripts used in the pipeline are listed below:

```
training_code/pdf_to_jsonl.py
training_code/train_tokenizer.py
training_code/tokenizer_diagnostics.py
training_code/retrieval/run_retrieval_demo.py
training_code/smoke_test.py
training_code/base_vs_peft.py
training_code/m3_dashboard.py
```

The reproducibility workflow included PDF ingestion, tokenizer training, tokenizer diagnostics, BM25 retrieval, smoke testing, base-versus-PEFT evaluation, plot generation, and dashboard execution.

VI. RESULTS

A. Tokenizer Results

The tokenizer performed well on the storm corpus diagnostic sample.

TABLE VIII
TOKENIZER DIAGNOSTIC RESULTS

Metric	Value
Number of documents	91
Average tokens per document	605.4835
Tokens per 1,000 characters	445.1941
Unknown-token rate	0.0

The most important result was the unknown-token rate of 0.0. This indicates that the tokenizer successfully represented the diagnostic corpus without generating unknown tokens, which is beneficial for downstream retrieval and language-model training.

B. Retrieval Results

BM25 retrieval returned relevant storm-response chunks for multiple test queries.

The highest score was observed for "flooding storm surge," suggesting strong keyword overlap between the query and relevant retrieval chunks. The remaining queries also produced meaningful scores, demonstrating practical retrieval performance for emergency-response search tasks.

TABLE IX
BM25 RETRIEVAL RESULTS

Query	Top-1 BM25 Score
flooding storm surge	8.6744
evacuation orders	4.9100
emergency kits water batteries	4.7686

C. LM Smoke-Test Results

The smoke-test training demonstrated a stable learning trend.

TABLE X
LM SMOKE-TEST LOSS TREND

Step	Train Loss	Validation Loss	Test Loss
0	5.7216	5.7109	5.6910
79	4.9590	4.9196	5.0266

Training loss decreased from 5.7216 to 4.9590, while validation loss decreased from 5.7109 to 4.9196. The test loss remained finite and stable, confirming that the tokenizer, batching pipeline, and training loop were functioning correctly.

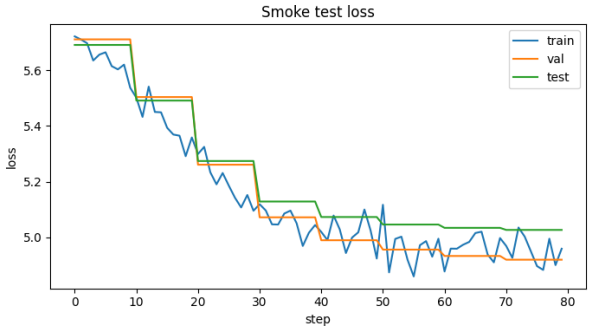

Fig. 2. Loss trend for the next-token language-model smoke test. The curve shows stable training behavior without exploding losses.

D. Base vs Adapted Comparison

The final experiment compared the fully trainable base model with the LoRA-adapted model.

TABLE XI
BASE VS LORA-ADAPTED MODEL COMPARISON

System	Trainable Parameters	Total Parameters	Test Loss	Test Perplexity
Base	470,528	470,528	4.8625	129.3472
Adapted LoRA	3,072	473,600	5.3904	219.2973

The base model achieved lower test loss and lower perplexity, indicating better held-out performance. However, the LoRA-adapted model trained only 3,072 parameters compared with 470,528 trainable parameters in the fully trainable base model. This corresponds to approximately 99.35% fewer trainable parameters.

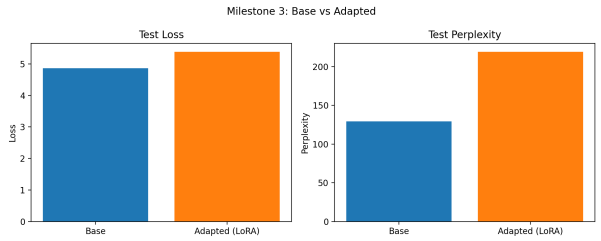

Fig. 3. Comparison of test perplexity between the base and LoRA-adapted models. The base model achieved lower perplexity, while LoRA provided substantial parameter efficiency.

VII. Analysis

A. Interpretation of Results

The results demonstrate that StormCare RAG successfully achieved its primary engineering objective of building a complete and reproducible storm-domain NLP workflow. The tokenizer achieved strong corpus coverage, BM25 retrieval returned relevant emergency-response passages, and the language-modeling pipeline trained without instability.

The fully trainable base model outperformed the LoRA-adapted model on held-out test loss and perplexity. However, this does not imply that LoRA is ineffective in general. In this project setting, the adaptation configuration was highly constrained. The LoRA model trained only a small subset of parameters and targeted only the `lm_head` layer for compatibility purposes, limiting its adaptation capacity.

B. What Worked Well

Several components of the system performed effectively, as summarized in Table XII.

TABLE XII
Components That Performed Well

Component	What Worked Well
PDF-to-JSONL pipeline	Created structured chunks from raw PDFs
Custom tokenizer	Achieved 0.0 unknown-token rate
BM25 retrieval	Returned relevant storm-response chunks
LM smoke training	Produced stable, non-exploding losses
Evaluation discipline	Used the same split, metrics, and evaluation procedure
Packaging workflow	Supported reproducible submission artifacts

C. What Did Not Work as Well

The LoRA-adapted model did not outperform the fully trainable base model. Several factors likely contributed to this outcome:

1) The model architecture was relatively compact.
2) The training budget was limited.
3) LoRA targeted only the `lm_head` layer.
4) The storm-domain corpus size was relatively small.
5) The adaptation objective may not have provided sufficient signal for stronger generalization improvements.

D. Representative Failure Cases and Difficult Examples

Failure Case 1: Retrieval over-prioritizing exact keywords.

A query such as "how should families prepare before landfall?" may fail to retrieve the best chunk if the corpus instead uses phrases such as "household readiness" or "pre-disaster planning."

Failure Case 2: Limited semantic retrieval capability.

BM25 performs effectively when query terms overlap with document vocabulary, but it may miss semantically related passages that use different wording.

Failure Case 3: LoRA underperformance in compact-model settings.

Although the LoRA-adapted model trained far fewer parameters, its limited adaptation capacity resulted in higher held-out perplexity compared with the fully trainable base model.

Failure Case 4: PDF extraction noise.

PDF files containing scanned pages, tables, unusual layouts, or formatting inconsistencies may produce noisy extracted text and lower-quality retrieval chunks.

Failure Case 5: Emergency-response interpretation risks.

Even when retrieval succeeds, retrieved passages alone may not be sufficient for real-world emergency decision-making. Official guidance and human oversight remain essential.

VIII. Limitations and Responsible Use

A. System Limitations

StormCare RAG has several important limitations.

First, the retrieval system uses BM25 only. While BM25 is interpretable and effective for keyword-based retrieval, it does not capture semantic similarity as effectively as dense or hybrid retrieval systems.

Second, the language model is compact and was trained under a limited computational budget. Therefore, the results should not be interpreted as performance claims for large-scale disaster-response language models.

Third, the LoRA configuration was intentionally narrow and targeted only the `lm_head` layer. A broader adaptation strategy involving attention or feed-forward layers may improve adaptation quality.

Fourth, the system does not implement a fully integrated retrieval-augmented generation (RAG) loop. Although retrieval and language modeling were demonstrated, retrieved evidence was not directly incorporated into generated responses.

Fifth, the PDF ingestion pipeline may struggle with scanned documents or image-based PDFs when optical character recognition (OCR) is unavailable. This can reduce extraction quality for emergency reports containing tables, graphics, or scanned pages.

B. Responsible Use

This system should not be deployed as an official emergency-response platform without extensive validation and expert oversight. Because disaster guidance can directly affect public safety, all outputs should be reviewed by qualified human experts before use in operational settings. The system is more appropriate for educational analysis, prototype development, document exploration, and research support rather than real-time emergency decision-making. Potential risks include outdated information, incomplete retrieval, hallucinated interpretations, and excessive confidence in generated outputs. To reduce these risks, any future deployed version should include source citations, uncertainty indicators, human review mechanisms, and references to official emergency-management agencies.

IX. Conclusion

This paper presented **StormCare RAG**, a compact and reproducible storm-domain NLP pipeline for PDF ingestion, tokenizer training, retrieval, next-token language modeling, and parameter-efficient adaptation analysis. The project demonstrated strong tokenizer coverage, effective BM25 retrieval, stable smoke-test training, and a controlled comparison between a fully trainable base model and a LoRA-adapted model.

The experimental results showed that the base model achieved lower held-out test loss and perplexity, while the LoRA configuration reduced trainable parameters by approximately 99.35%. These findings suggest that parameter-efficient adaptation can substantially reduce computational cost, although it does not necessarily improve model quality in compact-model and small-data settings.

Future work should explore hybrid retrieval methods, dense semantic embeddings, larger storm-domain corpora, expanded LoRA target modules, OCR-enabled PDF processing, and a fully integrated retrieval-augmented generation interface for source-grounded storm-response question answering.

References

[1] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[2] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.

[3] P. Lewis, E. Perez, A. Piktus, *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[4] E. J. Hu, Y. Shen, P. Wallis, *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," in *International Conference on Learning Representations (ICLR)*, 2022.

[5] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[6] StormCare RAG Project Documentation, "Milestone 2 + Milestone 3 Integrated Documentation," Project Result Guide, 2026.